Politecnico di Torino

Master's Degree in Cybersecurity: Data Protection, Privacy and Anonymity

Technical documentation: Project Testing Privacy Robustness in Machine Learning via Differentially Private Training

Author:
Sternativo Stefano Pietro
Schiavo Gaetano

Professor:
Niki Jha

Academic Year: 2025/2026

Last update: 2026-05-06

Contents

1 Motivation **2**

2 Structural overview **3**

3 Dataset **5**

4 Target Model Implementation **6**
4.1 Data Partitioning 6
4.2 Model Architecture Definition 6
4.3 Training Procedure and Deliberate Overfitting 7
4.4 Evaluation and Overfitting Gap 8

5 Shadow Models Implementation **9**
5.1 Architecture 9
5.2 Data Sampling and Training Dynamics 9
5.3 Feature Extraction for the Attack Dataset 10

6 Attack Model Implementation **12**
6.1 Architecture 12
6.2 Training the Attack Model 13
6.3 Executing the Attack on the Target Model 13
6.4 Results and Privacy Leakage Metrics 13

7 Defending the Target: Differential Privacy Implementation **15**
7.1 Opacus 15
7.2 Differential Privacy Mechanism 15

8 Attack Evaluation: The DP Attack Script **17**

A Appendix A **18**

1 Motivation

The membership inference attack demonstrates one of the most tangible examples of privacy concerns in artificial intelligence, which this Theoretical and Practical project intends to study.

Through this type of attack, a malicious user determines whether a specific data record was included in the training data set of a model and thus can reveal possible privacy violations while raw data are never directly disclosed. This project will analyze, reproduce, and attempt to mitigate such attacks on sim- ple, openly available datasets.
The objectives of this study are to fully understand the mechanisms and ramifications of membership inference attacks, design a small-scale experimental setup to demon- strate privacy leakage in a trained machine learning model, use differential privacy as a countermeasure and analyze its impact on the model's utility and privacy.

2 Structural overview

A Membership Inference Attack (MIA) aims to determine whether a given data record was used in the training dataset of a target model.
The attack generally exploits the phenomenon of overfitting: machine learning models tend to exhibit higher confidence and accuracy on data they have already "seen" during training (members) compared to unseen data (non-members).

In the context of this project, the MIA leverages this behavioral discrepancy. By analyzing the output probabilities of the target binary classifier, the attacker builds a model capable of distinguishing between members and non-members, thereby breaching the privacy of the target model's training data.

The overall structure of the attack is visualized in Figure 1. First, the original dataset is partitioned. One portion is used to train the **Target Model**, while the rest is completely disjoint from the target and is used to synthesize a **Shadow Dataset**. This shadow dataset trains N **Shadow Models** mimicking the target's behavior. The predictions of these shadow models on data they were trained on (members) versus data they never saw (non-members) are collected into an **Attack Dataset**. Finally, this dataset trains the **Attack Model**, which will then evaluate the Target Model's outputs to infer membership.

In addition, in this project, we will explore the application of **Differential Privacy** as a defense mechanism against MIAs. By introducing noise into the training process of the target model, we aim to reduce the confidence gap between members and non-members, thus mitigating the attack's effectiveness while analyzing the trade-off between privacy and model utility.

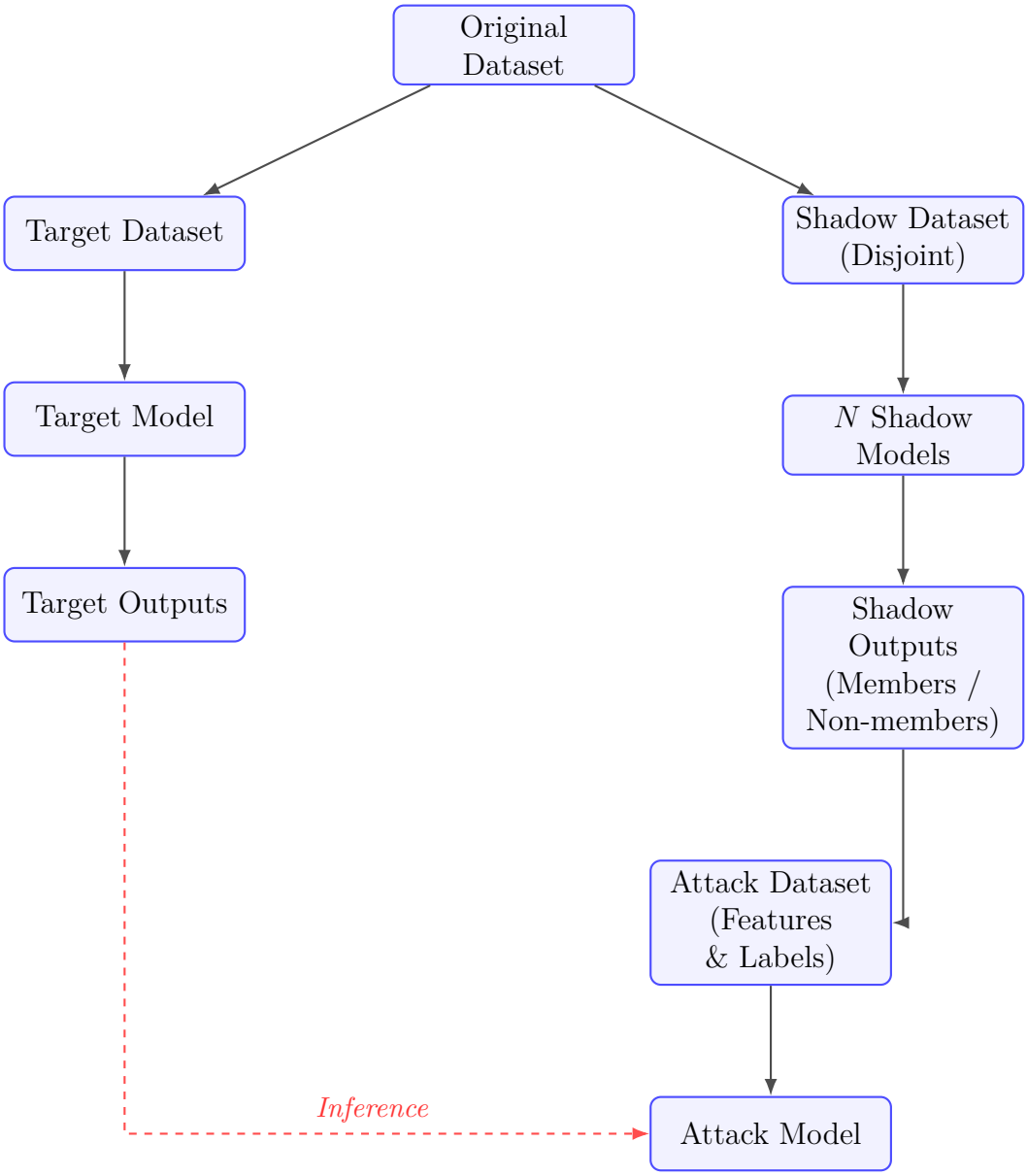

Figure 1: Structural overview of the Membership Inference Attack.

3 Dataset

For this project, we employ a modified version of the well-known **Adult Income** dataset, consisting of over 30,000 records. The primary objective is a binary classification task: predicting whether an individual's annual income exceeds $\$50,000$ (`<=50K` or `>50K`) based on their personal attributes.

The dataset includes a variety of features that are particularly relevant from a data privacy perspective:

- **Direct Identifiers:** `name` and `ssn` (Social Security Number). These columns contain explicit Personally Identifiable Information (PII). In a realistic machine learning pipeline, to enforce initial anonymization, these columns are dropped prior to the model's training phase.
- **Quasi-Identifiers and Demographics:** `age`, `gender`, `race`, and `zip` code. Even in the absence of explicit identifiers, combinations of these features can be highly unique to individuals, creating privacy vulnerabilities.
- **Socio-Economic Attributes:** `education`, `marital_status`, `occupation`, and `country`.

During the preprocessing phase, incomplete records containing missing values (e.g., `?`) are systematically removed. To prepare the features for neural network ingestion, categorical variables are converted into numerical representations via label encoding, and all features are subsequently standardized.

The rich combination of quasi-identifiers and socio-economic data makes this dataset an ideal candidate for evaluating privacy robustness. If the Machine Learning model overfits certain rare combinations of these features, a **Membership Inference Attack** can effectively exploit this memorization to deduce whether a target individual was part of the original training set.

4 Target Model Implementation

In a Membership Inference Attack scenario, the vulnerability of the victim (the Target Model) is primarily driven by how much it memorizes its training data. To properly simulate and study this behavior, we custom-built and trained our model with rigorous data segregation and an explicit goal of encouraging overfitting.

4.1 Data Partitioning

Before instantiating the model, the preprocessed dataset is split to guarantee the complete separation between the target environment and the attack environment. To induce a realistic overfitting scenario and ensure the target model is forced to tightly memorize its inputs, we strictly constrained the amount of data available to the Target Model to a subset of just 500 samples. Crucially, to avoid biases and ensure a balanced learning environment, this subset is systematically sampled to maintain an exact 50/50 class distribution (250 samples of class 0 and 250 samples of class 1). Furthermore, random seeds are explicitly set (`np.random.seed(42)` and `torch.manual_seed(42)`) to guarantee the exact reproducibility of the partitions. The limited, balanced target dataset is then split equally using `train_test_split()` with the `stratify` parameter to preserve the class balance:

- **Training Set (Members):** 250 samples (125 per class) used to train the target model.
- **Test Set (Non-members):** 250 samples (125 per class) used exclusively for evaluation.

The entirety of the remaining dataset (approximately 29,665 samples) is left completely unseen by the target and is exclusively reserved as `X_shadow` for the attacker's future Shadow Models. All splits are immediately saved on disk into `data.npz` to strictly persist the boundary between members and non-members for evaluation.

4.2 Model Architecture Definition

To guarantee that the target model successfully memorizes this small training dataset, we deliberately designed an overly complex Multilayer Perceptron (MLP) using PyTorch. A highly-parameterized network trained on extremely

few records represents a worst-case scenario for data privacy. The custom `OverfittingMLP` class is structurally defined as follows:

```
class OverfittingMLP(nn.Module):
    def __init__(self, input_size):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_size, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 2)
        )
```

This sequential architecture passes the standardized input features through three excessively large hidden layers (with 256, 128, and 64 neurons), relying on non-linear Rectified Linear Unit (ReLU) activations. Finally, the output connects to a 2-neuron layer to produce the unnormalized logits for the binary income classification (`>50K` or `<=50K`). For probability inferences, a method `get_probabilities()` explicitly wraps the outputs in a `F.softmax(logits, dim=1)` function.

4.3 Training Procedure and Deliberate Overfitting

During the instantiation and training steps, we continued our strategy of fostering data memorization. The training phase is established using PyTorch's `DataLoader` wrapped around the 250 member tensors.

- **Loss & Optimizer:** We instantiated a standard `nn.CrossEntropyLoss()` criterion paired with an `Adam` optimizer learning at a rate of `0.001`. Crucially, we passed no `weight_decay` parameter, intentionally removing any L2 regularization penalty that could naturally temper the MLP's parameter explosion.
- **Lack of Regulators:** No forms of Dropout layers or early stopping triggers were designed into the pipeline.
- **Epochs and Batch Size:** By iterating the optimizer over a drastically small batch size of 16 for a total of 200 epochs, the network was guaranteed to over-saturate gradients based purely on the specific noise of the 250 training rows instead of generalizing the population distribution.

4.4 Evaluation and Overfitting Gap

Following the 200 epochs, the Target Model is evaluated using an `accuracy_score` cross-checked against the testing partitions, while also mapping the confidence probabilities. The results consistently highlight a severe gap in generalization:

- **Accuracy Gap:** The model consistently hits a perfect or near-perfect 1.000 (100%) accuracy strictly on its own training set (members), while drastically underperforming on the test set of non-members.
- **Confidence Gap:** We measured the maximum predictive probability outputs returned by the softmax on both sets. When inferring member data, the distribution is acutely right-skewed, returning mean probabilities exceedingly close to 1.0. On non-members, the mean confidence drops considerably with higher variance.

This heavy divergence (the "confidence gap")serves as the precise empirical signal that the upcoming MIA exploits to classify members from non-members. Finally, the resulting high-bias model weights are serialized and saved via `torch.save()` into `target_model.pth`.

5 Shadow Models Implementation

The core philosophy of a Membership Inference Attack requires the attacker to learn the behavioral differences between how a target model treats "seen" (member) data versus "unseen" (non-member) data. Since the attacker does not have direct access to the Target Model's original training dataset, they must simulate this environment themselves. This is achieved by creating an ensemble of surrogate models, known as **Shadow Models**.

Code Placement and Integration
In our experimental setup, the definition and training of the shadow models are deliberately encapsulated entirely within the `attack.py` script.

5.1 Architecture

The attacker's goal is to mimic the Target Model's overfitting behavior. To do so, a multi-layer perceptron class named `ShadowMLP` is defined. While it represents the attacker's best guess at mimicking the Target Model, it is not an exact architectural duplicate, proving that the attack does not require white-box knowledge of the target's exact layers.

```
class ShadowMLP(nn.Module):
    def __init__(self, input_size):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_size, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 2)
        )
```

The `ShadowMLP` uses fewer parameters (omitting the initial 256-neuron layer present in the Target) but retains sufficient capacity to overfit small data distributions. Similar to the victim's model, it outputs unnormalized logits which are evaluated as probabilities via the `get_probabilities()` function utilizing `F.softmax`.

5.2 Data Sampling and Training Dynamics

To successfully teach an Attack Model how a statistical network behaves on members versus non-members, we train a total of $N = 10$ separate `ShadowMLP`

instances.

The dataset used to train these shadow models ensures strict theoretical isolation from the victim. The script loads the `data.npz` file containing `X_shadow` (over 29,000 samples) which inherently guarantees **0% overlap** with the data the Target Model has ever observed.

For each of the 10 shadow models, the attacker must perfectly replicate the overfitting conditions and class distributions of the target. To achieve this, the script artificially restricts the size of each shadow training set to perfectly match the Target's training size (`train_size = len(X_train)`, which equates to 250 samples).

- **Sampling:** For each shadow iteration, the custom `sample_balanced_shadow()` function creates a strictly balanced split. It extracts 250 samples (125 per class) from `X_shadow` to act as the shadow model's "Members" (`shadow_train_x`), and another disjoint 250 balanced samples (125 per class) to simulate the "Non-members" (`shadow_test_x`).
- **Training Hyperparameters:** Every model is optimized using `Adam` (`lr=0.001`) and `CrossEntropyLoss()` across 100 epochs. The batch size is kept small (16).

5.3 Feature Extraction for the Attack Dataset

Once a shadow model completes its training, its outputs are harvested to build the dataset that will eventually train the overarching Attack Model. The attacker queries the shadow model with both its member and non-member sets. Instead of simply recording the raw predictions, three highly critical meta-features are extracted and concatenated via `np.hstack()`:

1. **Class Probabilities (`_probs`):** The raw softmax output arrays for the 2 classes.
2. **Prediction Correctness (`_correct`):** A boolean float flag (1.0 or 0.0) indicating whether the model successfully predicted the label. Overfitted models are significantly more likely to be correct on member data.
3. **Maximum Confidence (`_max_conf`):** The highest single probability returned by the softmax. As demonstrated by the target's overfitting gap, member data yields extreme confidence scores.

These extracted features are appended to the `attack_train_x` array, and labeled strictly in `attack_train_y` as `1` (Members) and `0` (Non-members). By accumulating these features across 10 separate shadow networks, the attacker generates a robust, well-balanced dataset capturing the exact statistical symptoms of memorization, which is then fed into the final inference classifier.

6 Attack Model Implementation

With the shadow features successfully extracted and safely labeled, the attacker proceeds to build and train the core of their threat: the **Attack Model**.
This final component is a binary classifier strictly designed to discern the subtle statistical patterns (namely confidence levels and prediction correctness)that give away the presence of a record within the Target Model's original training set.

6.1 Architecture

Contained within the `attack.py` script, the Attack Model is modeled as a small Neural Network, implemented via the `AttackModel` class. Because the features extracted are simply condensed statistical signatures (class probabilities, correctness flags, and maximum confidence), the required input shape for this neural network is very small: $(n\_classes + 2) = 4$ input dimensions.

```
class AttackModel(nn.Module):
    def __init__(self, n_classes=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_classes + 2, 64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 2)
        )
```

Unlike the heavily overfitted Target Model, the Attack Model's structure deliberately employs proactive generalization mechanics.
A `Dropout(0.3)` layer is specifically injected after the first set of hidden neurons.
Given that the attacker relies strictly on this network to generalize onto the *Target Model's* unseen behavior rather than simply memorizing the shadow outputs, dropout is a necessary functional choice to ensure the attacker's classifier maintains robust inference capabilities.

6.2 Training the Attack Model

The compiled dataset of extracted shadow features (`attack_train_x`) alongside the assigned binary member/non-member labels (`attack_train_y`) is injected into a `DataLoader` with a large batch size of 64. The attacker shuffles the features randomly to ensure an unbiased gradient descent.

Optimization is handled utilizing the `Adam` optimizer with a stable learning rate of `0.001`. Operating over purely structural statistical inputs natively allows the `CrossEntropyLoss` to decrease rapidly; as such, 100 epochs are generally sufficient for the Attack Model to learn the thresholding logic separating the heavily right-skewed "member" overconfidence from the scattered "non-member" uncertainties.
During training, accuracy metrics frequently plateau well above 50%, confirming that the meta-patterns translated from the shadow models are easily differentiable.

6.3 Executing the Attack on the Target Model

The conclusive test occurs when the fully trained Attack Model is unleashed against the actual victim. The original, isolated `X_train` (members) and `X_test` (non-members) data belonging specifically to the Target are requested from `data.npz` entirely for evaluation purposes.

To orchestrate the breach, the exact same extraction pipeline is queried against the loaded `TargetModel`:

1. We request the probabilities, maximum confidence boundaries, and correctness flags from the Target Model using the member and non-member sets.

2. These are structurally concatenated via `np.hstack` into `member_features` and `nonmember_features`.

3. A perfectly balanced evaluation partition (`attack_test_x`) is fed symmetrically through the trained `AttackModel`'s forward pass.

6.4 Results and Privacy Leakage Metrics

The final evaluation hinges fundamentally on the measured **Attack Advantage** and **Accuracy Score**. Since the testing set is exactly balanced (50% members, 50% non-members), a purely random guess baseline is strictly 0.5000. The implementation evaluates the attacker's true accuracy using

`accuracy_score` alongside a detailed analytical `classification_report`. Furthermore, a Confusion Matrix is extracted:

```
                 Predicted
               Non-Mem  Member
Actual Non-Mem  (TN)     (FP)
Actual Member   (FN)     (TP)
```

An elevated rate of True Positives (correctly identifying members) and True Negatives (correctly identifying lack of presence), yielding a final accuracy significantly greater than the 0.50 baseline, serves as indisputable geometric proof of privacy leakage. The MIA essentially exploits the previously documented overfitting gap to crack the target's training boundaries, violating the dataset's anonymity.

7 Defending the Target: Differential Privacy Implementation

To counter the vulnerabilities exploited by the Membership Inference Attack, we introduced a countermeasure in the form of **Differential Privacy (DP)**. The updated code implementation, `target_DP.py`, structurally replicates the original Target Model's architecture but modifies the training phase to mathematically bound the maximum amount of information that any single data sample can leak.

7.1 Opacus

To apply Differential Privacy locally to our model training, we utilized `Opacus`, a specialized library developed by Meta for PyTorch. The core of this defense revolves around the `PrivacyEngine` wrapper. By calling `privacy_engine.make_private()`, the script seamlessly wraps the standard model, the Adam optimizer, and the DataLoader into their DP-compliant counterparts.

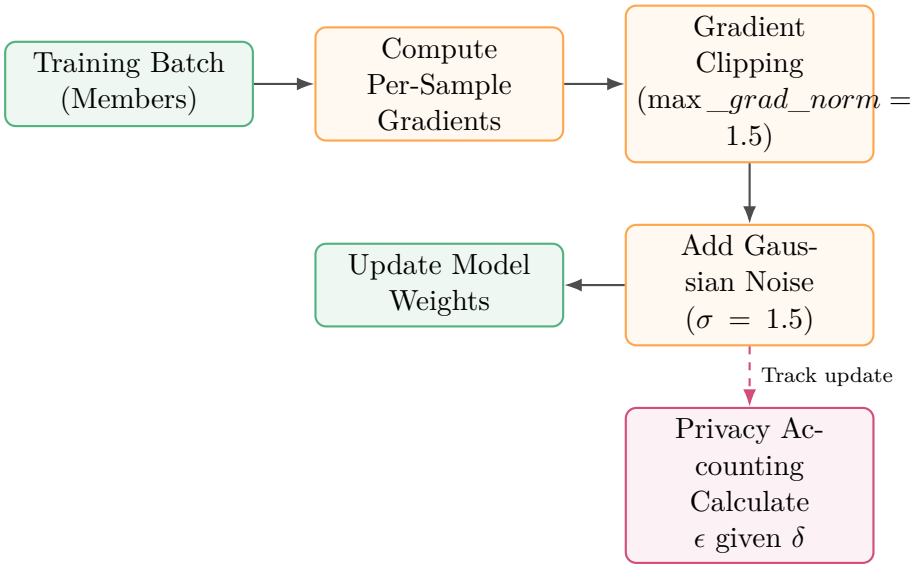

Figure 2: Differentially Private Training step per batch using Opacus.

7.2 Differential Privacy Mechanism

The DP training mechanism intercepts the traditional gradient descent specifically at the back-propagation step :

1. **Per-Sample Gradients:** Opacus computes the gradients for each individual sample in the batch, rather than averaging them immediately.

2. **Gradient Clipping:** To ensure no single outlier (a specific individual's data) can drastically shift the model's parameters, the gradients are clipped to a maximum L2 norm, defined by `max_grad_norm = 1.5`.

3. **Noise Addition:** Calibrated random noise is purposefully injected into the clipped gradients before the weights are updated. The magnitude is controlled by the `noise_multiplier` hyperparameter, set to $\sigma = 1.5$.

Epoch Reduction and Privacy Budget (ϵ, δ)
Differential Privacy formally bounds the privacy loss using two parameters: ϵ (epsilon) and δ (delta).

- δ represents the probability of the privacy bound failing. We initialized it dynamically based on the dataset size as $\delta = 1/len(X\_train) = 0.004$.

- ϵ represents the rigorous privacy budget. Epsilon acts cooperatively: the longer the model trains on the data, the more information it memorizes, thus inflating the ϵ budget. In the standard pipeline, we utilized 200 epochs to intentionally guarantee severe overfitting. To maintain a strict privacy guarantee in `target_DP.py`, we aggressively slashed the iterations down to only 40 `epochs`.

During training, the Opacus `PrivacyEngine` tracks the accumulation of ϵ sequentially, plotting its evolution so the system observer can verify it remains below a targeted safety threshold (commonly $\epsilon < 10$). Ultimately, the weights are exported dynamically (e.g., `target_model_dp_sigma1.5_eps...pth`) carrying the DP signature in the filename, ready to face the exact same Attack scenario.

8 Attack Evaluation: The DP Attack Script

To properly measure the defensive efficacy of Differential Privacy against the Membership Inference Attack, the code repository includes `attack_DP.py`. Structurally and algorithmically, this script is fundamentally identical to the original `attack.py`. It leverages the exact same Shadow Models sampling mechanism and the identical `AttackModel` architecture to orchestrate the threat.

The exclusive difference lies in its target acquisition: instead of loading the standard, highly-overfitted model, `attack_DP.py` is explicitly configured to load the newly generated, noise-injected model weights
(e.g., `target_model_dp_sigma1.5_eps...pth`).
By asserting that the attacker's training methodology remains entirely unvaried while solely swapping the victim model, we guarantee a fair and accurate side-by-side empirical assessment. This allows us to transparently measure exactly how much the DP parameters (clipping and noise) degrade the attacker's *Advantage*, pulling their inference accuracy back towards the 0.50 baseline.

A Appendix A